Capítulo 04 - Modelagem de Dados

4.1. Introdução à Modelagem de Dados

O Projeto Quimera utiliza uma arquitetura de **persistência poliglota**, onde diferentes tipos de dados são armazenados nos bancos de dados mais adequados para suas características e padrões de acesso. Esta abordagem maximiza a performance e a escalabilidade do sistema, permitindo que cada banco de dados seja otimizado para seu caso de uso específico.

4.1.1. Estratégia de Persistência Poliglota

PostgreSQL (Relacional) - Dados estruturados e transacionais

- Perfis, usuários, conversas, mensagens
- Relacionamentos complexos
- Queries analíticas
- Garantias ACID

MongoDB (Documento) - Dados semi-estruturados e flexíveis

- Análises completas de perfis (JSON grande)
- Logs estruturados
- Histórico de postagens
- Schema flexível

Redis (Chave-Valor) - Cache e filas

- Cache de queries frequentes
- Cache de embeddings
- Filas de mensagens (pub/sub)
- Session storage
- Rate limiting

ChromaDB (Vetorial) - Embeddings e busca semântica

- Embeddings de perfis
- Busca por similaridade
- Recomendação de perfis similares

4.1.2. Princípios de Modelagem

1. Normalização vs. Desnormalização

No PostgreSQL, seguimos a **3ª Forma Normal (3NF)** para dados transacionais, eliminando redundância e garantindo integridade. No entanto, para queries analíticas, criamos **views materializadas desnormalizadas** que agregam dados de múltiplas tabelas para acelerar leituras.

2. Indexação Estratégica

Criamos índices apenas para colunas frequentemente usadas em cláusulas WHERE, JOIN e ORDER BY. Índices em excesso degradam a performance de escritas.

3. Particionamento

Tabelas grandes (perfis, mensagens, logs) são particionadas por data para melhorar performance de queries e facilitar arquivamento de dados antigos.

4. Soft Delete

Em vez de deletar registros fisicamente, marcamos como deletados com uma flag `deleted_at`. Isso permite auditoria e recuperação de dados.

4.2. Schema PostgreSQL

4.2.1. Diagrama Entidade-Relacionamento (ER)

Plain Text

```
┌──────────────────┐
│    usuarios      │
└──────────────────┘
         │
         │
         │ 1:N
         │
┌────────▼──────────────────┐
│  preferencias_perfil      │
│  _ideal                   │
└───────────────────────────┘
         │
         │ 1:N
         │
┌────────▼──────────┐        ┌────────────────────┐
│  hotspots         │        │  perfis            │
│  _digitais        │        │                    │
└───────────────────┘        └────────────────────┘
         │                           │
         │                           │
         │ 1:N                       │ 1:N
```

```
          │                          │
          └──────────────────────────┘
                        │
                        │
                        │
          ┌─────────────▼────────────┐
          │  pontos_afinidade        │
          └──────────────────────────┘
                        │
                        │ N:1
                        │
          ┌─────────────▼────────────┐
          │  estrategias_persuasao   │
          └──────────────────────────┘
                        │
                        │
                        │ 1:1
                        │
          ┌─────────────▼────────────┐
          │  mensagens_geradas       │
          └──────────────────────────┘
                        │
                        │
                        │ 1:N
                        │
          ┌─────────────▼────────────┐
          │  conversas               │
          └──────────────────────────┘
                        │
                        │
                        │ 1:N
                        │
          ┌─────────────▼────────────┐
          │  mensagens               │
          └──────────────────────────┘
```

4.2.2. Tabelas Principais

Tabela: usuarios

SQL

```sql
-- Tabela de usuários do sistema
CREATE TABLE usuarios (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    email VARCHAR(255) UNIQUE NOT NULL,
    senha_hash VARCHAR(255) NOT NULL,
    nome_completo VARCHAR(255) NOT NULL,
    telefone VARCHAR(20),

    -- Dados pessoais para persona
```

```sql
    idade INTEGER,
    cidade VARCHAR(100),
    estado VARCHAR(2),
    interesses TEXT[],
    hobbies TEXT[],

    -- Status e permissões
    ativo BOOLEAN DEFAULT TRUE,
    email_verificado BOOLEAN DEFAULT FALSE,
    plano VARCHAR(50) DEFAULT 'free', -- 'free', 'basic', 'premium'

    -- Timestamps
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
    ultimo_login TIMESTAMP WITH TIME ZONE,
    deleted_at TIMESTAMP WITH TIME ZONE,

    -- Constraints
    CONSTRAINT usuarios_email_check CHECK (email ~* '^[A-Za-z0-9._%+-]+@[A-
Za-z0-9.-]+\.[A-Za-z]{2,}$'),
    CONSTRAINT usuarios_idade_check CHECK (idade >= 18 AND idade <= 100)
);

-- Índices
CREATE INDEX idx_usuarios_email ON usuarios(email) WHERE deleted_at IS NULL;
CREATE INDEX idx_usuarios_ativo ON usuarios(ativo) WHERE deleted_at IS NULL;
CREATE INDEX idx_usuarios_plano ON usuarios(plano) WHERE deleted_at IS NULL;

-- Trigger para atualizar updated_at
CREATE OR REPLACE FUNCTION update_updated_at_column()
RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = NOW();
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER usuarios_updated_at
    BEFORE UPDATE ON usuarios
    FOR EACH ROW
    EXECUTE FUNCTION update_updated_at_column();

-- Comentários
COMMENT ON TABLE usuarios IS 'Usuários do sistema Projeto Quimera';
COMMENT ON COLUMN usuarios.senha_hash IS 'Hash bcrypt da senha';
COMMENT ON COLUMN usuarios.plano IS 'Plano de assinatura: free, basic,
premium';
```

Tabela: preferencias_perfil_ideal

SQL

```sql
-- Preferências do perfil ideal do usuário
CREATE TABLE preferencias_perfil_ideal (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    usuario_id UUID NOT NULL REFERENCES usuarios(id) ON DELETE CASCADE,

    -- Pesos dos critérios (soma deve ser 1.0)
    peso_interessante DECIMAL(3,2) NOT NULL DEFAULT 0.40,
    peso_bonita DECIMAL(3,2) NOT NULL DEFAULT 0.30,
    peso_solteira DECIMAL(3,2) NOT NULL DEFAULT 0.30,

    -- Limiar de qualificação (0-10)
    limiar_qualificacao DECIMAL(3,1) NOT NULL DEFAULT 7.5,

    -- Critérios de "interessante"
    interesses_desejados TEXT[],
    hobbies_desejados TEXT[],
    personalidade_desejada TEXT[], -- ['extrovertida', 'aventureira',
'intelectual']

    -- Critérios de "bonita" (atributos estéticos)
    estilo_desejado TEXT[], -- ['casual', 'elegante', 'esportivo']

    -- Critérios de "solteira"
    prioridade_solteira BOOLEAN DEFAULT TRUE,

    -- Faixa etária
    idade_minima INTEGER DEFAULT 18,
    idade_maxima INTEGER DEFAULT 35,

    -- Localização
    cidade_alvo VARCHAR(100),
    raio_km INTEGER DEFAULT 50,

    -- Timestamps
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),

    -- Constraints
    CONSTRAINT preferencias_usuario_unique UNIQUE(usuario_id),
    CONSTRAINT preferencias_pesos_sum CHECK (peso_interessante + peso_bonita
+ peso_solteira = 1.0),
    CONSTRAINT preferencias_limiar_check CHECK (limiar_qualificacao >= 0 AND
limiar_qualificacao <= 10),
    CONSTRAINT preferencias_idade_check CHECK (idade_minima >= 18 AND
```

```
idade_maxima <= 100 AND idade_minima <= idade_maxima)
);

-- Índices
CREATE INDEX idx_preferencias_usuario ON
preferencias_perfil_ideal(usuario_id);

-- Trigger
CREATE TRIGGER preferencias_updated_at
    BEFORE UPDATE ON preferencias_perfil_ideal
    FOR EACH ROW
    EXECUTE FUNCTION update_updated_at_column();
```

Tabela: hotspots_digitais

SQL

```
-- Hotspots digitais para prospecção
CREATE TABLE hotspots_digitais (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    usuario_id UUID NOT NULL REFERENCES usuarios(id) ON DELETE CASCADE,

    -- Dados do hotspot
    nome VARCHAR(255) NOT NULL,
    tipo VARCHAR(50) NOT NULL, -- 'estabelecimento', 'evento',
'influenciador'
    plataforma VARCHAR(50) NOT NULL, -- 'instagram', 'facebook'
    url TEXT NOT NULL,
    username VARCHAR(100),

    -- Categoria
    categoria VARCHAR(100), -- 'bar', 'academia', 'restaurante', 'balada'

    -- Localização
    cidade VARCHAR(100),
    estado VARCHAR(2),
    latitude DECIMAL(10,8),
    longitude DECIMAL(11,8),

    -- Métricas
    total_seguidores INTEGER,
    total_perfis_coletados INTEGER DEFAULT 0,
    total_perfis_qualificados INTEGER DEFAULT 0,
    taxa_qualificacao DECIMAL(5,2), -- Percentual

    -- Status
    ativo BOOLEAN DEFAULT TRUE,
```

```sql
    prioridade INTEGER DEFAULT 5, -- 1-10

    -- Timestamps
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
    ultima_coleta TIMESTAMP WITH TIME ZONE,
    deleted_at TIMESTAMP WITH TIME ZONE,

    -- Constraints
    CONSTRAINT hotspots_url_unique UNIQUE(url),
    CONSTRAINT hotspots_plataforma_check CHECK (plataforma IN ('instagram',
'facebook')),
    CONSTRAINT hotspots_prioridade_check CHECK (prioridade >= 1 AND
prioridade <= 10)
);

-- Índices
CREATE INDEX idx_hotspots_usuario ON hotspots_digitais(usuario_id) WHERE
deleted_at IS NULL;
CREATE INDEX idx_hotspots_ativo ON hotspots_digitais(ativo) WHERE deleted_at
IS NULL;
CREATE INDEX idx_hotspots_plataforma ON hotspots_digitais(plataforma) WHERE
deleted_at IS NULL;
CREATE INDEX idx_hotspots_prioridade ON hotspots_digitais(prioridade DESC)
WHERE ativo = TRUE AND deleted_at IS NULL;

-- Trigger
CREATE TRIGGER hotspots_updated_at
    BEFORE UPDATE ON hotspots_digitais
    FOR EACH ROW
    EXECUTE FUNCTION update_updated_at_column();
```

Tabela: perfis (Particionada por Data)

SQL

```sql
-- Tabela principal de perfis (particionada)
CREATE TABLE perfis (
    id UUID DEFAULT gen_random_uuid(),
    usuario_id UUID NOT NULL REFERENCES usuarios(id) ON DELETE CASCADE,
    hotspot_origem_id UUID NOT NULL REFERENCES hotspots_digitais(id) ON
DELETE SET NULL,

    -- Dados básicos
    url TEXT NOT NULL,
    plataforma VARCHAR(50) NOT NULL,
    username VARCHAR(100),
```

```sql
    nome_completo VARCHAR(255),
    biografia TEXT,
    localizacao VARCHAR(255),
    foto_perfil_url TEXT,

    -- Métricas sociais
    seguidores INTEGER,
    seguindo INTEGER,
    total_postagens INTEGER,
    taxa_engajamento DECIMAL(5,2),

    -- Dados inferidos
    idade_aparente INTEGER,
    genero VARCHAR(20),

    -- Pontuações (0-10)
    pontuacao_interessante DECIMAL(3,1),
    pontuacao_bonita DECIMAL(3,1),
    pontuacao_solteira DECIMAL(3,1),
    pontuacao_final DECIMAL(3,1),
    confianca DECIMAL(3,2), -- 0-1
    ranking INTEGER,

    -- Status de processamento
    status_processamento VARCHAR(50) NOT NULL DEFAULT 'descoberto',
    -- 'descoberto', 'em_analise', 'qualificado', 'desqualificado',
    -- 'contatado', 'respondeu', 'handoff_realizado', 'finalizado'

    -- Timestamps
    timestamp_coleta TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
    timestamp_analise TIMESTAMP WITH TIME ZONE,
    timestamp_contato TIMESTAMP WITH TIME ZONE,
    timestamp_resposta TIMESTAMP WITH TIME ZONE,
    timestamp_handoff TIMESTAMP WITH TIME ZONE,
    timestamp_finalizacao TIMESTAMP WITH TIME ZONE,
    deleted_at TIMESTAMP WITH TIME ZONE,

    -- Constraints
    CONSTRAINT perfis_url_unique UNIQUE(url),
    CONSTRAINT perfis_plataforma_check CHECK (plataforma IN ('instagram',
'facebook')),
    CONSTRAINT perfis_pontuacao_check CHECK (
        (pontuacao_interessante IS NULL OR (pontuacao_interessante >= 0 AND
pontuacao_interessante <= 10)) AND
        (pontuacao_bonita IS NULL OR (pontuacao_bonita >= 0 AND
pontuacao_bonita <= 10)) AND
        (pontuacao_solteira IS NULL OR (pontuacao_solteira >= 0 AND
pontuacao_solteira <= 10)) AND
```

```
        (pontuacao_final IS NULL OR (pontuacao_final >= 0 AND
pontuacao_final <= 10))
    ),

    PRIMARY KEY (id, timestamp_coleta)
) PARTITION BY RANGE (timestamp_coleta);

-- Criar partições mensais (exemplo para 2025)
CREATE TABLE perfis_2025_01 PARTITION OF perfis
    FOR VALUES FROM ('2025-01-01') TO ('2025-02-01');

CREATE TABLE perfis_2025_02 PARTITION OF perfis
    FOR VALUES FROM ('2025-02-01') TO ('2025-03-01');

CREATE TABLE perfis_2025_03 PARTITION OF perfis
    FOR VALUES FROM ('2025-03-01') TO ('2025-04-01');

-- Criar partições automaticamente (PostgreSQL 14+)
CREATE OR REPLACE FUNCTION create_monthly_partition()
RETURNS void AS $$
DECLARE
    partition_date DATE;
    partition_name TEXT;
    start_date TEXT;
    end_date TEXT;
BEGIN
    -- Criar partição para o próximo mês
    partition_date := DATE_TRUNC('month', NOW() + INTERVAL '1 month');
    partition_name := 'perfis_' || TO_CHAR(partition_date, 'YYYY_MM');
    start_date := TO_CHAR(partition_date, 'YYYY-MM-DD');
    end_date := TO_CHAR(partition_date + INTERVAL '1 month', 'YYYY-MM-DD');

    EXECUTE format(
        'CREATE TABLE IF NOT EXISTS %I PARTITION OF perfis FOR VALUES FROM
(%L) TO (%L)',
        partition_name, start_date, end_date
    );
END;
$$ LANGUAGE plpgsql;

-- Agendar criação automática de partições (pg_cron)
SELECT cron.schedule('create-monthly-partition', '0 0 1 * *', 'SELECT
create_monthly_partition()');

-- Índices (serão criados em todas as partições)
CREATE INDEX idx_perfis_usuario ON perfis(usuario_id) WHERE deleted_at IS
NULL;
CREATE INDEX idx_perfis_hotspot ON perfis(hotspot_origem_id) WHERE
```

```
deleted_at IS NULL;
CREATE INDEX idx_perfis_status ON perfis(status_processamento) WHERE
deleted_at IS NULL;
CREATE INDEX idx_perfis_qualificados ON perfis(usuario_id, pontuacao_final
DESC)
    WHERE status_processamento = 'qualificado' AND deleted_at IS NULL;
CREATE INDEX idx_perfis_url ON perfis(url) WHERE deleted_at IS NULL;
CREATE INDEX idx_perfis_username ON perfis(username) WHERE deleted_at IS
NULL;

-- Índice GIN para busca full-text em biografia
CREATE INDEX idx_perfis_biografia_fts ON perfis USING
GIN(to_tsvector('portuguese', biografia))
    WHERE biografia IS NOT NULL AND deleted_at IS NULL;
```

Tabela: pontos_afinidade

SQL

```
-- Pontos de afinidade entre usuário e perfil
CREATE TABLE pontos_afinidade (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    perfil_id UUID NOT NULL,
    usuario_id UUID NOT NULL REFERENCES usuarios(id) ON DELETE CASCADE,

    -- Tipo de afinidade
    tipo VARCHAR(50) NOT NULL,
    -- 'interesse_comum', 'hobby_comum', 'local_comum', 'evento_comum',
'seguidor_comum'

    -- Descrição da afinidade
    descricao TEXT NOT NULL,

    -- Peso/importância (0-1)
    peso DECIMAL(3,2) NOT NULL DEFAULT 0.5,

    -- Dados específicos (JSONB para flexibilidade)
    detalhes JSONB,

    -- Timestamps
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),

    -- Constraints
    CONSTRAINT pontos_afinidade_perfil_tipo_unique UNIQUE(perfil_id, tipo,
descricao),
    CONSTRAINT pontos_afinidade_peso_check CHECK (peso >= 0 AND peso <= 1)
);
```

```sql
-- Índices
CREATE INDEX idx_pontos_afinidade_perfil ON pontos_afinidade(perfil_id);
CREATE INDEX idx_pontos_afinidade_usuario ON pontos_afinidade(usuario_id);
CREATE INDEX idx_pontos_afinidade_tipo ON pontos_afinidade(tipo);
CREATE INDEX idx_pontos_afinidade_peso ON pontos_afinidade(peso DESC);

-- Índice GIN para queries em JSONB
CREATE INDEX idx_pontos_afinidade_detalhes ON pontos_afinidade USING
GIN(detalhes);
```

Tabela: estrategias_persuasao

SQL

```sql
-- Estratégias de persuasão definidas pelo Motor de Persuasão
CREATE TABLE estrategias_persuasao (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    perfil_id UUID NOT NULL,
    usuario_id UUID NOT NULL REFERENCES usuarios(id) ON DELETE CASCADE,

    -- Princípio de persuasão primário (Cialdini)
    principio_primario VARCHAR(50) NOT NULL,
    -- 'afeicao', 'prova_social', 'reciprocidade', 'escassez', 'autoridade',
'compromisso_coerencia'

    -- Princípios secundários
    principios_secundarios TEXT[],

    -- Gatilhos emocionais
    gatilhos_emocionais TEXT[],

    -- Abordagem construída
    abordagem TEXT NOT NULL,

    -- Tom de comunicação
    tom VARCHAR(50) NOT NULL, -- 'casual', 'amigavel', 'confiante',
'prestativo'

    -- Pontos de afinidade a mencionar
    afinidades_mencionar UUID[], -- Array de IDs de pontos_afinidade

    -- Timestamps
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),

    -- Constraints
    CONSTRAINT estrategias_perfil_unique UNIQUE(perfil_id)
```

```
);

-- Índices
CREATE INDEX idx_estrategias_perfil ON estrategias_persuasao(perfil_id);
CREATE INDEX idx_estrategias_usuario ON estrategias_persuasao(usuario_id);
CREATE INDEX idx_estrategias_principio ON
estrategias_persuasao(principio_primario);
```

Tabela: mensagens_geradas

SQL

```
-- Mensagens geradas pelo Agente Escriba
CREATE TABLE mensagens_geradas (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    perfil_id UUID NOT NULL,
    usuario_id UUID NOT NULL REFERENCES usuarios(id) ON DELETE CASCADE,
    estrategia_id UUID NOT NULL REFERENCES estrategias_persuasao(id) ON
DELETE CASCADE,

    -- Conteúdo da mensagem
    conteudo TEXT NOT NULL,

    -- Metadados de geração
    modelo_llm VARCHAR(50) NOT NULL, -- 'gpt-4', 'gpt-4o-mini', 'claude-3-
opus'
    temperatura DECIMAL(3,2), -- Temperatura usada na geração
    tokens_usados INTEGER,
    custo_usd DECIMAL(10,6),

    -- Validação
    validada BOOLEAN DEFAULT FALSE,
    motivo_rejeicao TEXT,
    tentativa_numero INTEGER DEFAULT 1,

    -- Status
    status VARCHAR(50) NOT NULL DEFAULT 'gerada',
    -- 'gerada', 'aprovada', 'rejeitada', 'enviada'

    -- Timestamps
    timestamp_geracao TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
    timestamp_aprovacao TIMESTAMP WITH TIME ZONE,
    timestamp_envio TIMESTAMP WITH TIME ZONE,

    -- Constraints
    CONSTRAINT mensagens_perfil_unique UNIQUE(perfil_id),
    CONSTRAINT mensagens_conteudo_length CHECK (LENGTH(conteudo) <= 1000)
```

```
);

-- Índices
CREATE INDEX idx_mensagens_perfil ON mensagens_geradas(perfil_id);
CREATE INDEX idx_mensagens_usuario ON mensagens_geradas(usuario_id);
CREATE INDEX idx_mensagens_status ON mensagens_geradas(status);
CREATE INDEX idx_mensagens_modelo ON mensagens_geradas(modelo_llm);
```

Tabela: conversas

SQL

```
-- Conversas entre usuário e perfis
CREATE TABLE conversas (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    perfil_id UUID NOT NULL,
    usuario_id UUID NOT NULL REFERENCES usuarios(id) ON DELETE CASCADE,
    mensagem_inicial_id UUID REFERENCES mensagens_geradas(id) ON DELETE SET
NULL,
    estrategia_persuasao_id UUID REFERENCES estrategias_persuasao(id) ON
DELETE SET NULL,

    -- Status da conversa
    status VARCHAR(50) NOT NULL DEFAULT 'iniciada',
    -- 'iniciada', 'aguardando_resposta', 'respondida', 'handoff_pendente',
    -- 'handoff_realizado', 'finalizada', 'bloqueada'

    -- Timestamps
    timestamp_inicio TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
    timestamp_primeira_resposta TIMESTAMP WITH TIME ZONE,
    timestamp_handoff TIMESTAMP WITH TIME ZONE,
    timestamp_finalizacao TIMESTAMP WITH TIME ZONE,

    -- Métricas
    tempo_resposta_segundos INTEGER,
    numero_mensagens_bot INTEGER DEFAULT 0,
    numero_mensagens_humano INTEGER DEFAULT 0,
    numero_mensagens_alvo INTEGER DEFAULT 0,

    -- Resultado
    resultado VARCHAR(50), -- 'sucesso', 'falha', 'timeout', 'bloqueio'
    contato_obtido VARCHAR(100), -- WhatsApp, telefone, etc.
    notas TEXT,

    -- Constraints
    CONSTRAINT conversas_perfil_unique UNIQUE(perfil_id)
);
```

```sql
-- Índices
CREATE INDEX idx_conversas_perfil ON conversas(perfil_id);
CREATE INDEX idx_conversas_usuario ON conversas(usuario_id);
CREATE INDEX idx_conversas_status ON conversas(status);
CREATE INDEX idx_conversas_resultado ON conversas(resultado) WHERE resultado
IS NOT NULL;
CREATE INDEX idx_conversas_tempo_resposta ON
conversas(tempo_resposta_segundos) WHERE tempo_resposta_segundos IS NOT NULL;
```

Tabela: mensagens

SQL

```sql
-- Mensagens individuais dentro de conversas
CREATE TABLE mensagens (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    conversa_id UUID NOT NULL REFERENCES conversas(id) ON DELETE CASCADE,

    -- Remetente
    remetente VARCHAR(50) NOT NULL,
    -- 'agente_escriba', 'usuario_humano', 'alvo'

    -- Conteúdo
    conteudo TEXT NOT NULL,

    -- Análise de sentimento (para mensagens do alvo)
    sentimento VARCHAR(20), -- 'positivo', 'neutro', 'negativo'
    confianca_sentimento DECIMAL(3,2), -- 0-1

    -- Timestamps
    timestamp_envio TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
    timestamp_leitura TIMESTAMP WITH TIME ZONE,

    -- Constraints
    CONSTRAINT mensagens_remetente_check CHECK (remetente IN
('agente_escriba', 'usuario_humano', 'alvo'))
);

-- Índices
CREATE INDEX idx_mensagens_conversa ON mensagens(conversa_id,
timestamp_envio);
CREATE INDEX idx_mensagens_remetente ON mensagens(remetente);
CREATE INDEX idx_mensagens_sentimento ON mensagens(sentimento) WHERE
sentimento IS NOT NULL;
```

Tabela: episodios_treinamento

SQL

```sql
-- Episódios de treinamento para o Agente de Aprendizagem
CREATE TABLE episodios_treinamento (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    perfil_id UUID NOT NULL,
    usuario_id UUID NOT NULL REFERENCES usuarios(id) ON DELETE CASCADE,
    conversa_id UUID REFERENCES conversas(id) ON DELETE SET NULL,

    -- Estado (features do perfil)
    estado JSONB NOT NULL,

    -- Ação (pontuação atribuída, estratégia escolhida)
    acao JSONB NOT NULL,

    -- Recompensa (calculada com base no feedback)
    recompensa DECIMAL(5,3) NOT NULL, -- -1 a 1

    -- Próximo estado (resultado da interação)
    proximo_estado JSONB,

    -- Feedback do usuário
    avaliacao_perfil INTEGER, -- 1-5 estrelas
    avaliacao_mensagem INTEGER, -- 1-5 estrelas
    resultado_final VARCHAR(50), -- 'sucesso', 'falha'

    -- Timestamps
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),

    -- Constraints
    CONSTRAINT episodios_recompensa_check CHECK (recompensa >= -1 AND
recompensa <= 1),
    CONSTRAINT episodios_avaliacao_check CHECK (
        (avaliacao_perfil IS NULL OR (avaliacao_perfil >= 1 AND
avaliacao_perfil <= 5)) AND
        (avaliacao_mensagem IS NULL OR (avaliacao_mensagem >= 1 AND
avaliacao_mensagem <= 5))
    )
);

-- Índices
CREATE INDEX idx_episodios_perfil ON episodios_treinamento(perfil_id);
CREATE INDEX idx_episodios_usuario ON episodios_treinamento(usuario_id);
CREATE INDEX idx_episodios_recompensa ON episodios_treinamento(recompensa
DESC);
CREATE INDEX idx_episodios_resultado ON
```

```
episodios_treinamento(resultado_final) WHERE resultado_final IS NOT NULL;

-- Índice GIN para queries em JSONB
CREATE INDEX idx_episodios_estado ON episodios_treinamento USING GIN(estado);
CREATE INDEX idx_episodios_acao ON episodios_treinamento USING GIN(acao);
```

4.2.3. Views Materializadas

View: vw_perfis_qualificados

SQL

```
-- View materializada para perfis qualificados (otimização de leitura)
CREATE MATERIALIZED VIEW vw_perfis_qualificados AS
SELECT
    p.id,
    p.usuario_id,
    p.hotspot_origem_id,
    h.nome AS hotspot_nome,
    p.url,
    p.plataforma,
    p.username,
    p.nome_completo,
    p.biografia,
    p.localizacao,
    p.foto_perfil_url,
    p.seguidores,
    p.seguindo,
    p.total_postagens,
    p.pontuacao_interessante,
    p.pontuacao_bonita,
    p.pontuacao_solteira,
    p.pontuacao_final,
    p.confianca,
    p.ranking,
    p.status_processamento,
    p.timestamp_coleta,
    p.timestamp_analise,
    COUNT(pa.id) AS total_afinidades,
    ARRAY_AGG(pa.tipo) FILTER (WHERE pa.tipo IS NOT NULL) AS tipos_afinidade,
    e.principio_primario,
    e.tom AS tom_comunicacao,
    mg.conteudo AS mensagem_gerada,
    mg.status AS status_mensagem
FROM perfis p
LEFT JOIN hotspots_digitais h ON h.id = p.hotspot_origem_id
LEFT JOIN pontos_afinidade pa ON pa.perfil_id = p.id
```

```sql
LEFT JOIN estrategias_persuasao e ON e.perfil_id = p.id
LEFT JOIN mensagens_geradas mg ON mg.perfil_id = p.id
WHERE p.status_processamento IN ('qualificado', 'contatado', 'respondeu')
  AND p.deleted_at IS NULL
GROUP BY p.id, h.nome, e.principio_primario, e.tom, mg.conteudo, mg.status
ORDER BY p.pontuacao_final DESC, p.timestamp_analise DESC;

-- Índices na view materializada
CREATE UNIQUE INDEX idx_vw_perfis_qualificados_id ON
vw_perfis_qualificados(id);
CREATE INDEX idx_vw_perfis_qualificados_usuario ON
vw_perfis_qualificados(usuario_id, pontuacao_final DESC);
CREATE INDEX idx_vw_perfis_qualificados_status ON
vw_perfis_qualificados(status_processamento);

-- Refresh automático a cada 5 minutos
CREATE OR REPLACE FUNCTION refresh_vw_perfis_qualificados()
RETURNS void AS $$
BEGIN
    REFRESH MATERIALIZED VIEW CONCURRENTLY vw_perfis_qualificados;
END;
$$ LANGUAGE plpgsql;

SELECT cron.schedule('refresh-perfis-qualificados', '*/5 * * * *',
    'SELECT refresh_vw_perfis_qualificados()');
```

View: vw_metricas_dashboard

SQL

```sql
-- View materializada para métricas do dashboard
CREATE MATERIALIZED VIEW vw_metricas_dashboard AS
SELECT
    u.id AS usuario_id,
    u.nome_completo,

    -- Métricas de perfis
    COUNT(DISTINCT p.id) AS total_perfis_coletados,
    COUNT(DISTINCT p.id) FILTER (WHERE p.status_processamento =
'qualificado') AS total_perfis_qualificados,
    COUNT(DISTINCT p.id) FILTER (WHERE p.status_processamento =
'desqualificado') AS total_perfis_desqualificados,
    ROUND(AVG(p.pontuacao_final) FILTER (WHERE p.pontuacao_final IS NOT
NULL), 2) AS pontuacao_media,

    -- Métricas de conversas
    COUNT(DISTINCT c.id) AS total_conversas_iniciadas,
```

```sql
    COUNT(DISTINCT c.id) FILTER (WHERE c.status = 'respondida') AS
total_respostas_recebidas,
    COUNT(DISTINCT c.id) FILTER (WHERE c.resultado = 'sucesso') AS
total_sucessos,
    ROUND(
        COUNT(DISTINCT c.id) FILTER (WHERE c.status = 'respondida')::DECIMAL
/
        NULLIF(COUNT(DISTINCT c.id), 0) * 100,
        2
    ) AS taxa_resposta_pct,
    ROUND(
        COUNT(DISTINCT c.id) FILTER (WHERE c.resultado = 'sucesso')::DECIMAL
/
        NULLIF(COUNT(DISTINCT c.id) FILTER (WHERE c.status = 'respondida'),
0) * 100,
        2
    ) AS taxa_conversao_pct,

    -- Métricas de tempo
    ROUND(AVG(c.tempo_resposta_segundos) FILTER (WHERE
c.tempo_resposta_segundos IS NOT NULL) / 3600, 2) AS
tempo_medio_resposta_horas,

    -- Métricas de custo
    SUM(mg.custo_usd) AS custo_total_llm_usd,

    -- Última atividade
    MAX(p.timestamp_coleta) AS ultima_coleta,
    MAX(c.timestamp_inicio) AS ultima_conversa

FROM usuarios u
LEFT JOIN perfis p ON p.usuario_id = u.id AND p.deleted_at IS NULL
LEFT JOIN conversas c ON c.usuario_id = u.id
LEFT JOIN mensagens_geradas mg ON mg.usuario_id = u.id
WHERE u.deleted_at IS NULL
GROUP BY u.id, u.nome_completo;

-- Índices
CREATE UNIQUE INDEX idx_vw_metricas_dashboard_usuario ON
vw_metricas_dashboard(usuario_id);

-- Refresh automático a cada 10 minutos
SELECT cron.schedule('refresh-metricas-dashboard', '*/10 * * * *',
    'REFRESH MATERIALIZED VIEW CONCURRENTLY vw_metricas_dashboard');
```

4.3. Schema MongoDB

4.3.1. Coleções e Estruturas

Coleção: analises_perfis

JavaScript

```javascript
// Análise completa de um perfil (documento grande e flexível)
{
  _id: ObjectId("..."),
  perfil_id: "uuid-do-perfil",
  usuario_id: "uuid-do-usuario",

  // Dados básicos (duplicados do PostgreSQL para facilitar queries)
  url: "https://instagram.com/username",
  username: "username",
  nome_completo: "Nome Completo",

  // Análise textual (NLP)
  analise_textual: {
    biografia: {
      texto_original: "Biografia do perfil...",
      texto_limpo: "biografia perfil",
      tokens: ["biografia", "perfil"],
      entidades: [
        { tipo: "PESSOA", texto: "Nome", confianca: 0.95 },
        { tipo: "LOCAL", texto: "Varginha", confianca: 0.89 }
      ],
      interesses_extraidos: ["viagem", "fotografia", "musica"],
      hobbies_extraidos: ["yoga", "leitura"],
      personalidade_inferida: {
        extroversao: 0.75,
        abertura: 0.82,
        conscienciosidade: 0.65,
        amabilidade: 0.78,
        neuroticismo: 0.35
      },
      sentimento_geral: {
        polaridade: 0.65, // -1 a 1
        subjetividade: 0.45 // 0 a 1
      }
    },

    postagens: [
      {
        id: "post-123",
```

```
        legenda: "Texto da legenda...",
        hashtags: ["#viagem", "#praia"],
        mencoes: ["@amiga"],
        localizacao: "Praia de Copacabana",
        timestamp: ISODate("2025-10-01T14:30:00Z"),
        curtidas: 150,
        comentarios: 25,
        sentimento: {
          polaridade: 0.8,
          emocao_dominante: "alegria"
        }
      }
    ]
  },

  // Análise visual (Computer Vision)
  analise_visual: {
    foto_perfil: {
      url: "https://...",
      faces_detectadas: 1,
      face_principal: {
        bbox: [100, 100, 200, 200],
        idade_aparente: 28,
        genero: "feminino",
        confianca_genero: 0.92,
        emocao: "feliz",
        confianca_emocao: 0.78,
        atributos_esteticos: {
          cabelo_cor: "castanho",
          cabelo_comprimento: "longo",
          oculos: false,
          sorriso: true
        }
      },
      contexto: "retrato",
      estilo: "casual",
      qualidade: 0.85
    },

    postagens_fotos: [
      {
        id: "post-123",
        url: "https://...",
        faces_detectadas: 2,
        objetos_detectados: ["praia", "ceu", "mar"],
        contexto: "viagem",
        localizacao_inferida: "praia",
        timestamp: ISODate("2025-10-01T14:30:00Z")
```

```
      }
    ],

    analise_agregada: {
      contextos_frequentes: ["viagem", "festa", "academia"],
      estilo_predominante: "casual",
      frequencia_selfies: 0.35,
      frequencia_fotos_grupo: 0.45,
      frequencia_fotos_viagem: 0.60
    }
  },

  // Inferência de status de relacionamento
  inferencia_relacionamento: {
    status_inferido: "solteira",
    confianca: 0.75,
    evidencias: [
      {
        tipo: "ausencia_parceiro_recorrente",
        peso: 0.4,
        descricao: "Nenhuma pessoa recorrente em fotos"
      },
      {
        tipo: "hashtags",
        peso: 0.2,
        descricao: "Uso de #solteira em postagem recente"
      },
      {
        tipo: "linguagem_biografia",
        peso: 0.15,
        descricao: "Biografia não menciona relacionamento"
      }
    ],
    sinais_contrarios: []
  },

  // Pontuações finais
  pontuacoes: {
    interessante: 8.5,
    bonita: 7.8,
    solteira: 8.2,
    final: 8.2,
    confianca: 0.82
  },

  // Embeddings (para busca semântica)
  embeddings: {
    biografia_embedding: [0.123, -0.456, 0.789, ...], // 384 dimensões
```

```
    perfil_completo_embedding: [0.234, -0.567, 0.890, ...] // 384 dimensões
  },

  // Metadados de processamento
  metadata: {
    timestamp_analise: ISODate("2025-10-10T16:30:00Z"),
    tempo_processamento_segundos: 45.3,
    modelos_utilizados: {
      nlp: "neuralmind/bert-base-portuguese-cased",
      vision: "openai/clip-vit-base-patch32",
      sentimento: "cardiffnlp/twitter-xlm-roberta-base-sentiment"
    },
    versao_pipeline: "1.2.3"
  }
}
```

Índices MongoDB:

JavaScript

```javascript
// Índices para queries frequentes
db.analises_perfis.createIndex({ perfil_id: 1 }, { unique: true });
db.analises_perfis.createIndex({ usuario_id: 1 });
db.analises_perfis.createIndex({ "pontuacoes.final": -1 });
db.analises_perfis.createIndex({ "metadata.timestamp_analise": -1 });

// Índice composto para queries de perfis qualificados
db.analises_perfis.createIndex({
  usuario_id: 1,
  "pontuacoes.final": -1
});

// Índice de texto para busca full-text
db.analises_perfis.createIndex({
  "analise_textual.biografia.texto_original": "text",
  "analise_textual.postagens.legenda": "text"
}, {
  weights: {
    "analise_textual.biografia.texto_original": 10,
    "analise_textual.postagens.legenda": 5
  },
  name: "text_search_index"
});
```

Coleção: logs_sistema

JavaScript

```
// Logs estruturados do sistema
{
  _id: ObjectId("..."),
  timestamp: ISODate("2025-10-10T16:30:00Z"),
  nivel: "INFO", // DEBUG, INFO, WARNING, ERROR, CRITICAL
  agente: "agente_analista",
  acao: "analisar_perfil",

  // Contexto
  perfil_id: "uuid-do-perfil",
  usuario_id: "uuid-do-usuario",
  correlation_id: "req-789xyz",

  // Mensagem
  mensagem: "Perfil analisado com sucesso",

  // Dados adicionais
  dados: {
    pontuacao_final: 8.5,
    tempo_processamento_ms: 1234,
    memoria_usada_mb: 512
  },

  // Erro (se houver)
  erro: {
    tipo: "ValueError",
    mensagem: "Valor inválido",
    stack_trace: "..."
  }
}
```

Índices:

JavaScript

```
db.logs_sistema.createIndex({ timestamp: -1 });
db.logs_sistema.createIndex({ nivel: 1, timestamp: -1 });
db.logs_sistema.createIndex({ agente: 1, timestamp: -1 });
db.logs_sistema.createIndex({ correlation_id: 1 });

// TTL index para expirar logs antigos (30 dias)
db.logs_sistema.createIndex(
  { timestamp: 1 },
  { expireAfterSeconds: 2592000 }
);
```

4.4. Schema Redis

4.4.1. Estruturas de Dados

Cache de Perfis

Plain Text

```
# String: Cache de perfil completo (JSON serializado)
Key: perfil:{perfil_id}
Value: {"id": "...", "nome": "...", ...}
TTL: 3600 segundos (1 hora)

# Exemplo
SET perfil:abc123 '{"id":"abc123","nome":"Maria Silva",...}' EX 3600
GET perfil:abc123
```

Cache de Embeddings

Plain Text

```
# String: Cache de embedding de perfil
Key: embedding:perfil:{perfil_id}
Value: [0.123, -0.456, 0.789, ...]  (JSON array)
TTL: 86400 segundos (24 horas)

# Exemplo
SET embedding:perfil:abc123 '[0.123,-0.456,0.789,...]' EX 86400
GET embedding:perfil:abc123
```

Filas de Eventos (Lists)

Plain Text

```
# List: Fila de perfis para análise
Key: fila:perfis_para_analise
Value: Lista de perfil_ids

# Exemplo
LPUSH fila:perfis_para_analise "abc123"
RPOP fila:perfis_para_analise  # Retorna "abc123"
LLEN fila:perfis_para_analise  # Tamanho da fila
```

Pub/Sub para Eventos

Plain Text

```
# Canal: Eventos de perfis analisados
Canal: eventos:PerfilAnalisado

# Publisher
PUBLISH eventos:PerfilAnalisado '{"perfil_id":"abc123","pontuacao":8.5}'

# Subscriber
SUBSCRIBE eventos:PerfilAnalisado
```

Rate Limiting

Plain Text

```
# String: Contador de ações por conta
Key: rate_limit:conta:{conta_id}:acao:{acao}
Value: Contador de ações
TTL: 86400 segundos (24 horas)

# Exemplo: Limitar envio de mensagens a 20 por dia
INCR rate_limit:conta:conta123:acao:enviar_mensagem
EXPIRE rate_limit:conta:conta123:acao:enviar_mensagem 86400
GET rate_limit:conta:conta123:acao:enviar_mensagem  # Retorna contador atual
```

Session Storage

Plain Text

```
# Hash: Dados de sessão do usuário
Key: sessao:{session_id}
Value: Hash com dados da sessão
TTL: 3600 segundos (1 hora)

# Exemplo
HSET sessao:sess123 usuario_id "user456" nome "João" email "joao@example.com"
EXPIRE sessao:sess123 3600
HGETALL sessao:sess123
```

4.5. Schema ChromaDB

4.5.1. Coleções de Embeddings

Coleção: perfis_embeddings

Python

```python
# Criar coleção
import chromadb

client = chromadb.Client()

collection = client.create_collection(
    name="perfis_embeddings",
    metadata={
        "description": "Embeddings de perfis para busca semântica",
        "embedding_model": "sentence-transformers/paraphrase-multilingual-
mpnet-base-v2",
        "embedding_dimension": 768
    }
)

# Adicionar embeddings
collection.add(
    embeddings=[
        [0.123, -0.456, 0.789, ...],  # 768 dimensões
        [0.234, -0.567, 0.890, ...]
    ],
    documents=[
        "Biografia do perfil 1...",
        "Biografia do perfil 2..."
    ],
    metadatas=[
        {
            "perfil_id": "abc123",
            "usuario_id": "user456",
            "pontuacao_final": 8.5,
            "timestamp": "2025-10-10T16:30:00Z"
        },
        {
            "perfil_id": "def456",
            "usuario_id": "user456",
            "pontuacao_final": 7.8,
            "timestamp": "2025-10-10T17:00:00Z"
        }
    ],
    ids=["abc123", "def456"]
)
```

```
# Buscar perfis similares
results = collection.query(
    query_embeddings=[[0.123, -0.456, 0.789, ...]],
    n_results=10,
    where={"usuario_id": "user456", "pontuacao_final": {"$gte": 7.5}}
)
```

4.6. Estratégias de Otimização

4.6.1. Particionamento de Tabelas

Tabelas grandes como `perfis` e `mensagens` são particionadas por data para melhorar performance de queries e facilitar arquivamento.

Benefícios:

- Queries mais rápidas (scan apenas partições relevantes)
- Manutenção mais fácil (drop de partições antigas)
- Melhor gerenciamento de índices

4.6.2. Indexação Estratégica

Criamos índices apenas para colunas frequentemente usadas em:

- Cláusulas WHERE
- JOINs
- ORDER BY

Índices Compostos:

SQL

```sql
CREATE INDEX idx_perfis_usuario_pontuacao
ON perfis(usuario_id, pontuacao_final DESC)
WHERE status_processamento = 'qualificado';
```

Índices Parciais:

SQL

```sql
CREATE INDEX idx_perfis_qualificados
ON perfis(pontuacao_final DESC)
```

```
WHERE status_processamento = 'qualificado' AND deleted_at IS NULL;
```

4.6.3. Views Materializadas

Views materializadas pré-computam queries complexas, acelerando leituras.

Refresh Automático:

```SQL
SELECT cron.schedule('refresh-view', '*/5 * * * *',
    'REFRESH MATERIALIZED VIEW CONCURRENTLY vw_perfis_qualificados');
```

4.6.4. Cache em Redis

Dados frequentemente acessados são cacheados em Redis com TTL apropriado.

Estratégia de Cache:

- Perfis: 1 hora
- Embeddings: 24 horas
- Métricas de dashboard: 10 minutos

4.7. Conclusão do Capítulo

Este capítulo cobriu a modelagem completa de dados do Projeto Quimera:

- **PostgreSQL:** 12 tabelas principais com particionamento, índices e views materializadas
- **MongoDB:** 2 coleções com documentos flexíveis e índices otimizados
- **Redis:** Estruturas para cache, filas, pub/sub e rate limiting
- **ChromaDB:** Coleção de embeddings para busca semântica
- **Otimizações:** Particionamento, indexação estratégica, views materializadas e cache

No próximo capítulo, mergulharemos na implementação do **Agente Batedor**, o primeiro agente do pipeline.

Próximo Capítulo: Capítulo 06 - Agente Batedor (Implementação Completa)

Referências deste Capítulo:

- PostgreSQL Documentation: https://www.postgresql.org/docs/
- MongoDB Documentation: https://www.mongodb.com/docs/

- Redis Documentation: https://redis.io/documentation
- ChromaDB Documentation: https://docs.trychroma.com/